ADM Project Assigment

- In this project, the **Referencing** approach was chosen for the MongoDB document model. Persons, companies, and works_in relationships are stored as separate collections. This matches the **Many-to-Many** nature of the **WORKS_IN** relationship: one person can work in multiple companies and one company can employ multiple persons. Embedding was not chosen because embedding company data inside each person document would cause data duplication and make updates inconsistent
- `Technical stack` **:**
  - RDBMS: **PosgtreSQL**
  - DDBMS: **MongoDB**
  - GDBMS: **Neo4j**
  - Language: **Python** (**Drives: Pg8000, PyMongo, Neo4j**)
  - Processing: ApacheSpark (**PySpark**)
- `Project structure` **:**

  Relational vs Document models
  - **Data entities:**
    - Person (id, name, surname, birth, address, country)
    - Company (id, name, founding date, sector)
  - **Relationship:**
    - A person works in 0 or more companies (ManyToMany+Optional)
  - **Core tasks:**
    - RDBMS Queries: Implement 4 types of SQL joins to match persons and companies
    - Spark+MongoDB: Use DataFrames or SparkSQL to perform the same matching operations

        - Spark RDD (Resilient Distributed Dataset): Implement the same logic using only specific RDD functions (map, flatmap, ReduceByKey, GroupByKey)

Relational vs Graph Models

- **Data entities:**
    - Social network user profiles and follow relationships
- **Core tasks:**
    - GDBMS Implementation: Use Cypher (Neo4j) to find all users via transitive follow connections
    - Spark + RDBMS: Import relational data into Spark and use Graphframes to find reachable users
    - Spark RDD: Re-implement the transitive reachability logic using only RDD specific functions

- There are 2 saving relation types in Documentary DBMS:
    1. `Embedding`
        - **Self-contained logic**: It put all the relational information inside a single document. There is only one record that contains everything. Logic:
            - Data is "Owned". The data inside belongs specifically to the parent. For example:
                - User's birthdate or address
            - No independence:
                - The sub-data (like address) usually doesn't need to exist on its own without person
            - Optimal for:
                - One-to-Flow: When a person has only a few times (person having 2-3 phone numbers)

- High-performance:
    - When we want to get all info in one shot. We don't want the DB to look in two different places

2. `Reference`

- **Link-Pointer logic:** It keep data in seperate collections. One document just holds the ID (reference of the document). Logic:
    - **Independence:** Both entities are important on their own.
    - **Many to Many:** One person works in many companies and are company has many employees
    - **Optimal for:**
        - **Frequent updates**: If company changes it's name,you only change it one place (the company collection). All the person documents stay the same because they only hold the ID
        - **Large data:** If company has 10.000 employees, we can't embed all of them into one company document because MongoDB has 16MB limit per document
    - **Data reusable:**
        - The same data needs to be shared accross many different records without many thousands of copies

`Graph model (Neo4j)`

- **Transactive closure** (Chain Link). **[ObjectA → ObjectB → ObjectC]**. All of these related to each other
    - **Direct closure**: ObjectA → ObjectB (A is directly related to B)
    - **Transactive closure:** ObjectA→ObjectB→ObjectC. A is not directly to B, but it is related to its through C.
    - If we use RDBMS or DBMS in this case: we can not handle it with big data.

- **In SQL**: We need to JOIN in every step. If 10 relations, 10 JOIN. Recruicive queries eat up a lot of memory.
- **In NoSQL**: We cannot find an object just one query.
  - For example: Want to find ObjectC, we have to find firstly ObjectA, then ObjectB. This causes N+1 query problem.
  - Example query: We have a Linkedin system's Database system:
    - We have one user, his name is "Steve". And he has a over 100 millions of friends, subscribes, connections. If we want to his followers, we can use:

```
MATCH (Steve) - [:FOLLOWS*] -> (others) RETURN others
```

- This DB systems optimal for:
  1. Social networks
  2. Recomendation engines
  3. Fraud detection (bank transactions)
- GraphQL systems have 2 main methods:
  - Node → (Objects, values): USER
  - Edge → (Connections): Follow, Connections, Friends, Subscribers

Project system structure

```
advanced-data-project/

├── config/
│   ├── settings.py
│
├── db/
│   ├── postgres.sql
│   ├── mongo_seed.py
```

```
│   └── neo4j.cypher
│
├── spark_jobs/
│   ├── rdd_job.py
│   ├── mongo_rdd_job.py
│   ├── mongo_df_job.py
│   └── performance_test.py
│
├── performance_test.py
├── requirements.txt
└── .env
└── README.md
```

Code part [RDBMS]:

`Environment Setup` :

- **Install Python, Psql** → Coding part. **VS Code** → Code environment

```
-- ==========================================================
======================
-- Advanced Data Management project (Assigment)
-- This file implements the RELATIONAL (RDBMS) side of the
assignment.
--
-- PART 1 — Comparing relational vs document models:
--   Schema + sample data: PERSON, COMPANY, WORK_IN (slide
conceptual model).
--   Queries below map to Part 1 Tasks 1–4 (SQL join varia
nts).
--   Extra aggregate queries at end of Part 1: supporting
analytics only (not
--   separate numbered tasks on the slide).
--
-- PART 2 — Comparing relational vs graph models:
```

```
--   Schema + sample data: USER, FOLLOW (relational side o
f the social network).
--   Graph queries live in db/neo4j.cypher; Spark graph jo
bs read these tables.
-- ==========================================================
======================

-- PART 1 — Relational schema: PERSON, COMPANY, WORK_IN (s
lide entity classes)
CREATE TABLE person(
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    surname VARCHAR(100) NOT NULL,
    birthdate DATE,
    address TEXT,
    country VARCHAR(100)
);

CREATE TABLE company(
    id SERIAL PRIMARY KEY,
    name VARCHAR(200) NOT NULL,
    founded DATE,
    sector VARCHAR(50)
);

CREATE TABLE works_in(
    person_id INTEGER REFERENCES person(id),
    company_id INTEGER REFERENCES company(id),
    PRIMARY KEY(person_id, company_id)
);

-- Sectors must match assignment: automotive, banking, ser
vices, healthcare, chemicals, public
INSERT INTO company (name, sector, founded)
VALUES
('Google',   'services',   '1998-09-04'),
```

```
('Amazon',   'services',   '1994-07-05'),
('Meta',     'services',   '2004-02-04'),
('Tesla',    'automotive', '2003-07-01'),
('OpenAI',   'services',   '2015-12-11');


INSERT INTO person (name, surname, birthdate, address, cou
ntry)
VALUES
('Mark','Zuckerberg','1984-10-10','California','US'),
('Pavel','Durov','1983-10-10','Moscow','Russia'),
('Sam','Altman','1985-04-22','California','US'),
('Elon','Musk','1971-06-28','Texas','US'),
('Larry','Page','1973-03-26','California','US'),
('Sergey','Brin','1973-08-21','California','US');


INSERT INTO works_in (person_id, company_id)
VALUES
(1,3),  -- Mark → Meta
(2,3),  -- Pavel → Meta
(3,5),  -- Sam → OpenAI
(4,4),  -- Elon → Tesla
(5,1),  -- Larry → Google
(6,1),  -- Sergey → Google
(3,1);  -- Sam → Google (multi-company)

-- PART 1, Slide Task 1 — All employed persons + company t
hey work in (INNER JOIN)
SELECT
    p.name,
    p.surname,
    c.name AS company_name,
    c.sector
FROM person p
JOIN works_in w ON p.id = w.person_id
```

```
JOIN company c ON w.company_id = c.id;

-- PART 1, Slide Task 2 — All persons + company informatio
n (if any) (LEFT JOIN)
SELECT
    p.name,
    p.surname,
    c.name AS company_name
FROM person p
LEFT JOIN works_in w ON p.id = w.person_id
LEFT JOIN company c ON w.company_id = c.id;

-- PART 1, Slide Task 3 — All companies + persons they emp
loy (if any)
-- Base table must be company so employers with zero emplo
yees still appear.
SELECT
    p.name,
    p.surname,
    c.name AS company_name
FROM company c
LEFT JOIN works_in w ON c.id = w.company_id
LEFT JOIN person p ON w.person_id = p.id;


-- PART 1, Slide Task 4 — All persons and companies; match
employed persons to companies
-- Chained FULL OUTER on person->works->company loses corr
ect NULL pairing; use persons+orphan companies.
WITH employed AS (
    SELECT p.name, p.surname, c.name AS company_name
    FROM person p
    LEFT JOIN works_in w ON p.id = w.person_id
    LEFT JOIN company c ON w.company_id = c.id
),
orphan_companies AS (
```

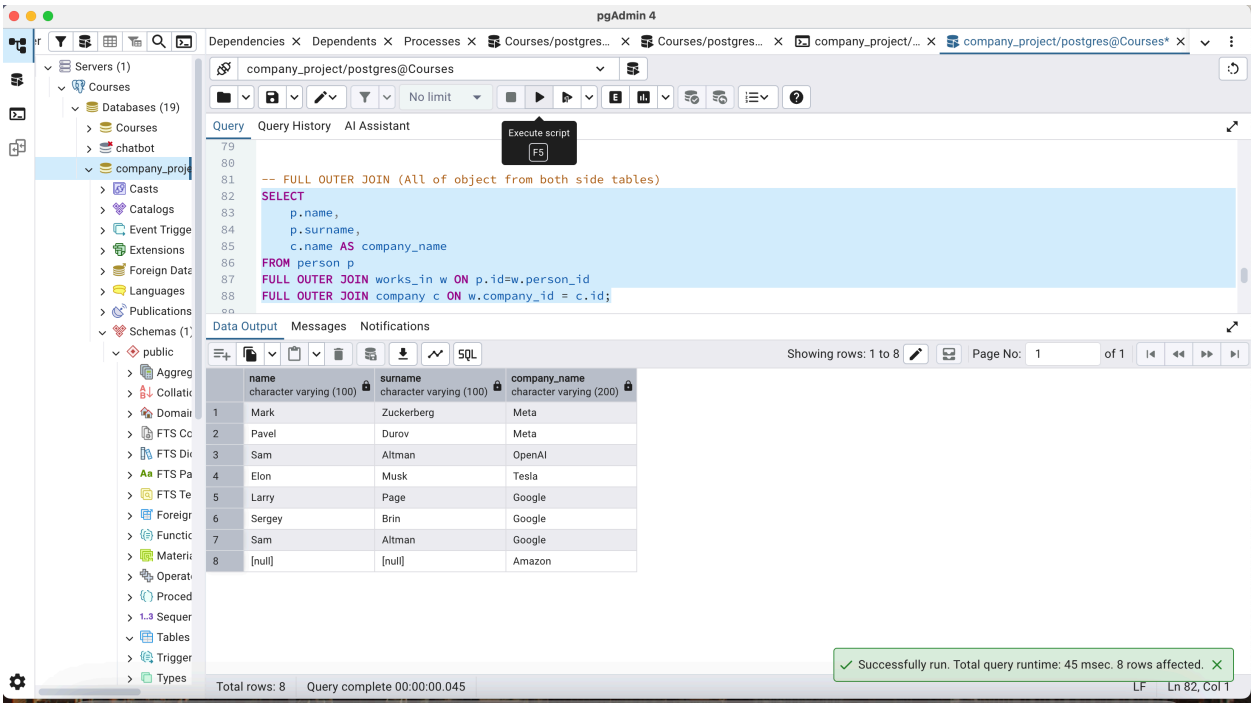
```
    SELECT NULL::varchar AS name, NULL::varchar AS surnam
e, c.name AS company_name
    FROM company c
    WHERE NOT EXISTS (SELECT 1 FROM works_in w WHERE w.com
pany_id = c.id)
)
SELECT name, surname, company_name FROM employed
UNION ALL
SELECT name, surname, company_name FROM orphan_companies;

-- PART 1 — Supplementary (not a separate slide task): per
son ↔ company counts
-- All persons with number of companies
SELECT
    p.name,
    COUNT(w.company_id) AS companies_count
FROM person p
LEFT JOIN works_in w ON p.id = w.person_id
GROUP by p.name;

-- All companies with number of persons
SELECT
    c.name,
    COUNT(w.company_id) AS employees_count
FROM company c
LEFT JOIN works_in w ON c.id = w.company_id
GROUP by c.name;
```

- **Task 4 requires matching all persons and all companies, including unemployed persons and companies with no employees. A direct chained FULL OUTER JOIN on person → works_in → company produces incorrect NULL pairings for orphan companies. Instead, a WITH clause (CTE) is used: first all persons are matched to their companies via LEFT JOIN, then orphan companies (those not present in works_in) are added via UNION ALL. This correctly represents the full outer semantics**

DML

```
-- ==========================================================
======================
-- PART 2 — Relational side: USER + FOLLOW (Assigment Part
2)
-- Implements the slide USER entity (name, surname, birthd
ate, address, country,
-- optional job, optional study_title) and FOLLOW relation
ships.
-- GDBMS / Spark programs use Neo4j + Spark; they load fro
m these tables where noted.
-- ==========================================================
======================

CREATE TABLE users (
    id      SERIAL PRIMARY KEY,
    name    VARCHAR(100) NOT NULL,
    surname VARCHAR(100) NOT NULL,
    birthdate DATE,
    address TEXT,
    country VARCHAR(100),
    job     VARCHAR(100),
    study_title VARCHAR(100)
);

CREATE TABLE follows (
    follower_id INTEGER REFERENCES users(id),
    followee_id INTEGER REFERENCES users(id),
    PRIMARY KEY (follower_id, followee_id)
);
INSERT INTO users (name, surname, birthdate, address, coun
try, job, study_title)
VALUES
('Mark','Zuckerberg','1984-10-10','Palo Alto, Californi
a','US','CEO','Harvard dropout'),
```

```
('Pavel','Durov','1984-10-10', 'Dubai','UAE', 'CEO','Saint
Petersburg State'),
('Sam','Altman','1985-04-22', 'San Francisco, Californi
a','US','CEO','Stanford'),
('Elon','Musk','1971-06-28', 'Austin, Texas','US','CEO','P
enn/Stanford'),
('Larry','Page','1973-03-26', 'Palo Alto, California','U
S',  NULL, 'Stanford PhD'),
('Sergey', 'Brin','1973-08-21', 'Palo Alto, California','U
S',NULL,'Stanford PhD'),
('Jeff','Bezos','1964-01-12', 'Seattle, Washington','U
S','Founder','Princeton'),
('Sundar','Pichai','1972-07-10','Mountain View, Californi
a','US','CEO','IIT Kharagpur');

INSERT INTO follows (follower_id, followee_id)
VALUES
(1, 2),
(2, 3),
(3, 4),
(4, 5),
(5, 6),
(1, 7),
(7, 8),
(6, 8);
```



- **Task 3 requires all companies to appear in the result, even those with no employees. For this reason, the base table is `company`, and a LEFT JOIN is used toward `works_in` and `person`. This ensures companies with zero employees still appear with NULL person fields**

Coding part [DDBMS-Mongo]:

`Environment setup` :

- **db/mongo_seed.py**

```
"""
Assigment – PART 1, document (DDBMS) model.

Loads sample PERSON / COMPANY / WORK_IN-style collections
into MongoDB.
Same conceptual data as the relational Part 1 side (postgr
es.sql), stored as documents.
```

```
Used by:
  - Part 1, Slide Tasks 5–8 — Spark jobs reading from Mong
o (mongo_df_job, mongo_sql_job).
  - Part 1, Slide Tasks 9–12 — Spark RDD jobs (mongo_rdd_j
ob).
  - services/mongo_service.py — API join-style aggregation
s (mirror Tasks 5–8 semantics).
"""
from pymongo import MongoClient
import sys
from pathlib import Path

ROOT_DIR = Path(__file__).resolve().parents[1]
if str(ROOT_DIR) not in sys.path:
    sys.path.append(str(ROOT_DIR))

from config.settings import MONGO_DB, MONGO_URI

client = MongoClient(MONGO_URI)
db = client[MONGO_DB]

# Clean old data
db.persons.delete_many({})
db.companies.delete_many({})
db.works_in.delete_many({})

persons = [
    {"person_id": 1, "name": "Mark",   "surname": "Zuckerb
erg"},
    {"person_id": 2, "name": "Pavel",  "surname": "Duro
v"},
    {"person_id": 3, "name": "Sam",    "surname": "Altma
n"},
    {"person_id": 4, "name": "Elon",   "surname": "Musk"},
    {"person_id": 5, "name": "Larry",  "surname": "Page"},
    {"person_id": 6, "name": "Sergey", "surname": "Brin"},
```

```
]
db.persons.insert_many(persons)

companies = [
    {"company_id": 1, "name": "Google",  "sector": "Tec
h"},
    {"company_id": 2, "name": "Amazon",  "sector": "E-comm
erce"},
    {"company_id": 3, "name": "Meta",    "sector": "Socia
l"},
    {"company_id": 4, "name": "Tesla",   "sector": "Automo
tive"},
    {"company_id": 5, "name": "OpenAI",  "sector": "AI"},
]
db.companies.insert_many(companies)

works_in = [
    {"person_id": 1, "company_id": 3},
    {"person_id": 2, "company_id": 3},
    {"person_id": 3, "company_id": 5},
    {"person_id": 3, "company_id": 1},
    {"person_id": 4, "company_id": 4},
    {"person_id": 5, "company_id": 1},
    {"person_id": 6, "company_id": 1},
]
db.works_in.insert_many(works_in)

print("MongoDB seed successfully inserted")
```

- **Tasks 5–8 are implemented using Spark DataFrames reading from MongoDB. Data is loaded via PyMongo into Python lists, then converted to Spark DataFrames. The four join types (inner, left, right, full outer) are performed using Spark's built-in `.join()` method with the corresponding join type parameter. DataFrames were chosen over raw RDDs for Tasks**

5–8 because Spark's Catalyst optimizer automatically optimizes DataFrame operations, resulting in cleaner and more efficient code.

Coding part [GDBMS-Neo4j]:

`Environment setup` :

- https://neo4j.com/download/
  1. New Project
  2. New Database
  3. username: `neo4j`
  4. password: `password` (eslab qol ❗)
  5. START

  http://localhost:7474

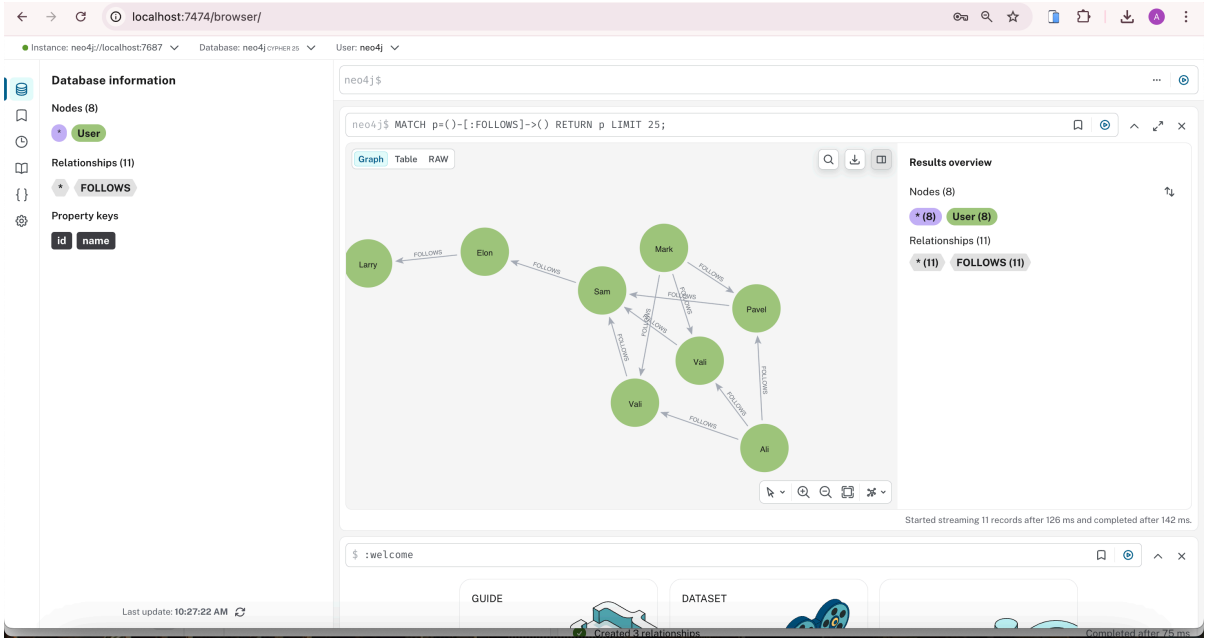
db/neo4j.cypher:

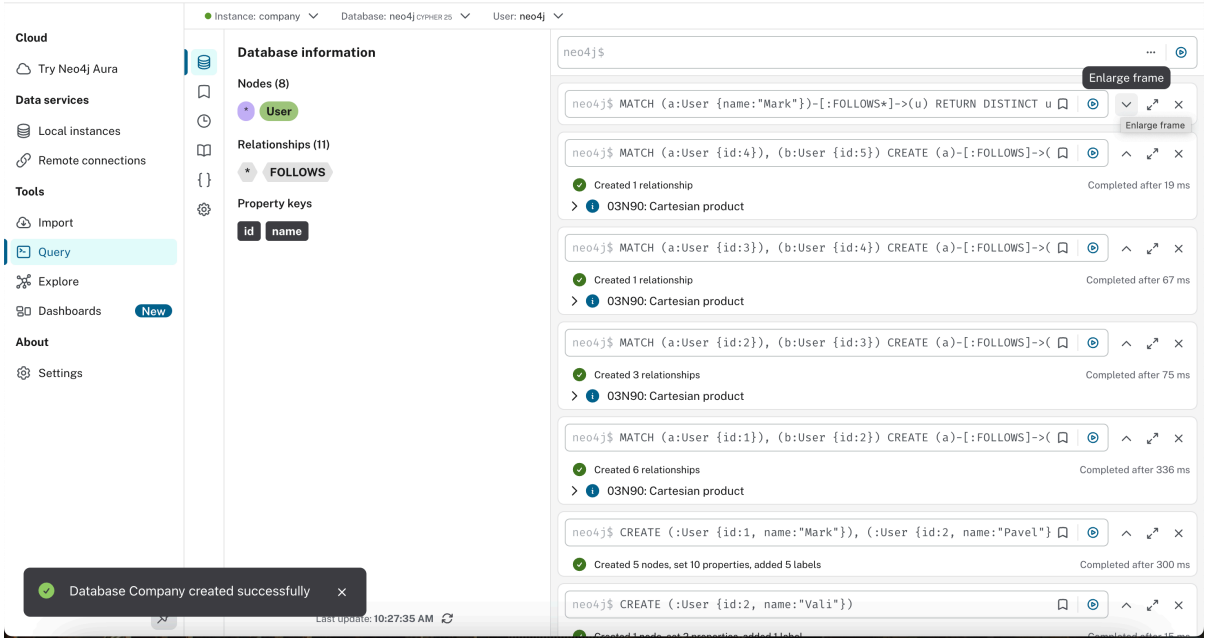
```
// ==========================================================
======================
// Advanced Data Management
// PART 2 — GRAPH (GDBMS) side: USER nodes + FOLLOWS relat
ionships.
// Implements the slide USER model (optional job, study_ti
tle) and FOLLOW edges.
//
// Sample graph below supports:
//   Part 2, Slide Task 1 — transitive “who can see whose
content” (example query at file end).
// Spark Part 2 Task 2 (GraphFrames) and Task 3 (RDD) load
the same network from PostgreSQL
// in jobs; this file is the canonical Neo4j dataset for T
ask 1 + API demo.
// ==========================================================
```

```
=======================

// Clean all existing nodes and relationships
MATCH (n) DETACH DELETE n;

// Sample USER nodes (slide Part 2 conceptual model)
CREATE
(:User {
    id: 1,
    name: "Mark",
    surname: "Zuckerberg",
    birthdate: "1984-10-10",
    address: "Palo Alto, California",
    country: "US",
    job: "CEO",
    study_title: "Harvard dropout"
}),
(:User {
    id: 2,
    name: "Pavel",
    surname: "Durov",
    birthdate: "1984-10-10",
    address: "Dubai",
    country: "UAE",
    job: "CEO",
    study_title: "Saint Petersburg State"
}),
(:User {
    id: 3,
    name: "Sam",
    surname: "Altman",
    birthdate: "1985-04-22",
    address: "San Francisco, California",
    country: "US",
    job: "CEO",
    study_title: "Stanford"
```

```
}),
(:User {
    id: 4,
    name: "Elon",
    surname: "Musk",
    birthdate: "1971-06-28",
    address: "Austin, Texas",
    country: "US",
    job: "CEO",
    study_title: "Penn/Stanford"
}),
(:User {
    id: 5,
    name: "Larry",
    surname: "Page",
    birthdate: "1973-03-26",
    address: "Palo Alto, California",
    country: "US",
    study_title: "Stanford PhD"
}),
(:User {
    id: 6,
    name: "Sergey",
    surname: "Brin",
    birthdate: "1973-08-21",
    address: "Palo Alto, California",
    country: "US",
    study_title: "Stanford PhD"
}),
(:User {
    id: 7,
    name: "Jeff",
    surname: "Bezos",
    birthdate: "1964-01-12",
    address: "Seattle, Washington",
    country: "US",
```

```
    job: "Founder",
    study_title: "Princeton"
}),
(:User {
    id: 8,
    name: "Sundar",
    surname: "Pichai",
    birthdate: "1972-07-10",
    address: "Mountain View, California",
    country: "US",
    job: "CEO",
    study_title: "IIT Kharagpur"
});

MATCH (a:User {id:1}), (b:User {id:2}) CREATE (a)-[:FOLLOW
S]->(b);
MATCH (a:User {id:2}), (b:User {id:3}) CREATE (a)-[:FOLLOW
S]->(b);
MATCH (a:User {id:3}), (b:User {id:4}) CREATE (a)-[:FOLLOW
S]->(b);
MATCH (a:User {id:4}), (b:User {id:5}) CREATE (a)-[:FOLLOW
S]->(b);
MATCH (a:User {id:5}), (b:User {id:6}) CREATE (a)-[:FOLLOW
S]->(b);
MATCH (a:User {id:1}), (b:User {id:7}) CREATE (a)-[:FOLLOW
S]->(b);
MATCH (a:User {id:7}), (b:User {id:8}) CREATE (a)-[:FOLLOW
S]->(b);
MATCH (a:User {id:6}), (b:User {id:8}) CREATE (a)-[:FOLLOW
S]->(b);

// PART 2, Slide Task 1 – Given user "Mark", all users who
se posts/content he can see transitively
MATCH (a:User {name:"Mark"})-[:FOLLOWS*]->(u)
RETURN DISTINCT u.name AS reachable_user;
```





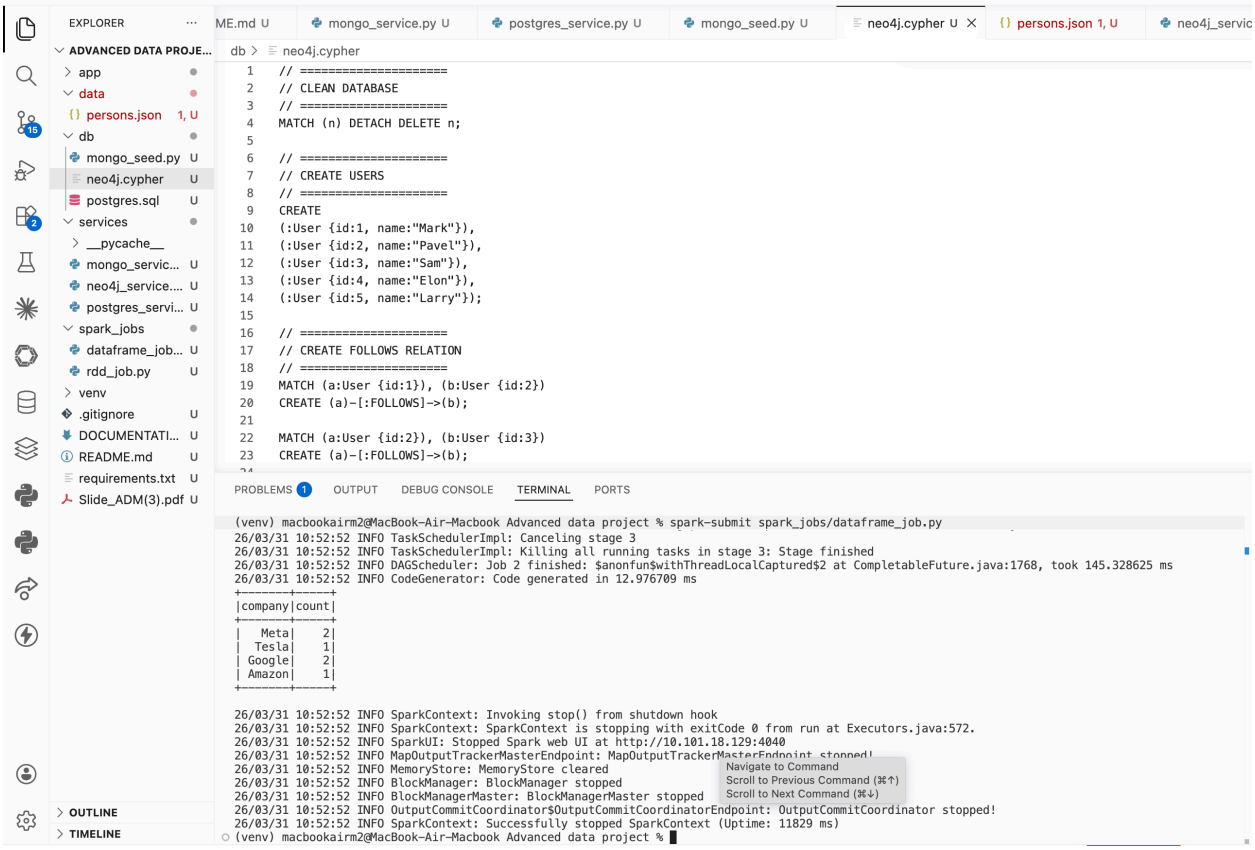
Running:

```
Terminal:

python3 db/mongo_seed.py
```

```
mongosh
```

`Spark DataFrame` :

```
spark-submit spark_jobs/rdd_job.py

spark-submit spark_jobs/dataframe_job.py
```



- **Tasks 9–12 re-implement the same four join semantics using only the permitted RDD functions: map, flatMap, filter, groupByKey, distinct, union, subtract, isEmpty, and collect. The RDD join() function was intentionally not used, as it is not in the permitted list. Instead, lookup dictionaries were built using map() and collect(), and join logic was manually implemented using map() and filter(). For the RIGHT JOIN (Task 11), groupByKey() was used to**

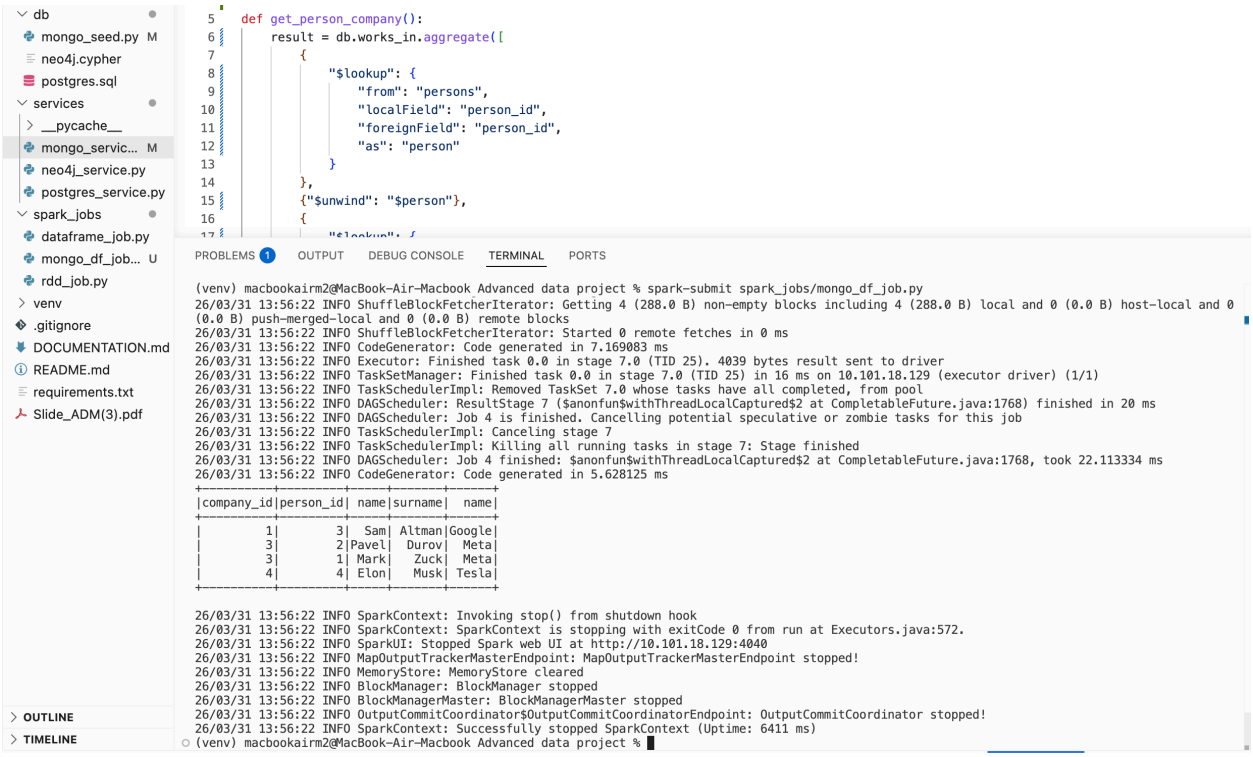
group persons by company. For the FULL OUTER JOIN (Task 12), the LEFT JOIN result was combined with unmatched companies using union().

- **Apache spark** is a distributed computing framework used to process large-scale data efficiently. Instead of processing data on a single machines, Spark distributes the workload accross multiple machines and processes data in parallel.
- **Why do we need spark?**
  - **Problem:**
    - When working with large-data (GBs, TBs):
      - Single machine is too slow
      - Memory (RAM) is not enough
      - Processing may crash
      - Scaling is limited

    (**Example**: Processing 1TB data with Python → very slow or impossible with kind a loop functions)
  - **Solution:**
    - Spark involves this by:
      - Splitting data into smaller parts
      - Distributing data accross machines
      - Processing everything in parallel
    - Result:
      - Faster and parallel computation
      - Scalable system
      - Fault-tolerant execution

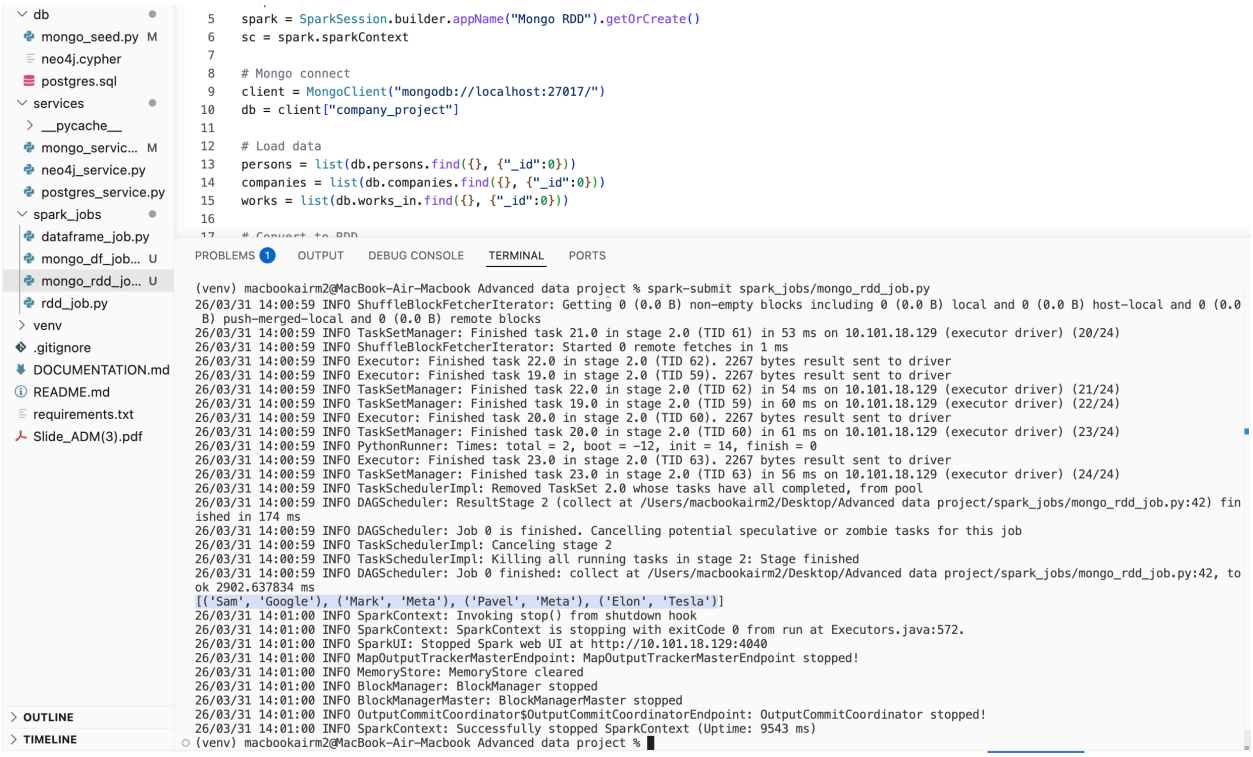
    (**Example**: 100GB → 10 machines → each processes 10GB with parallel)

- **Spark architecture:**
  - **Components:**
    1. **Driver: (main program, controls execution, sends tasks)**
    2. **Executors: (worker nodes, execute tasks, process data)**
    3. **Cluster: (group of machines, runs the distributed system)**
- **RDD** (**Resilient Distributed Dataset**):
  - RDD is the core data structure in Spark. It is: ca
    - Distributed collection of data
    - Immutable (cannot be changed)
    - Processed in parallel
  - **Why RDD?**
    - Full control over computation
    - Fault-tolerant
    - Flexible
  - **RDD operations:**
    - Transformations:
      - map()
      - filter()
      - flatMap()
    - Actions:
      - collect()
      - count()
      - reduce()
- `Spark Dataframe: RDD with JOIN in Mongo`



RDD provides low-level control and allows manual implementation of MapReduce operations such as map, shuffle, and reduce

- **Task 2 of Part 2 is implemented using GraphFrames. User and follow data is loaded from PostgreSQL into Spark DataFrames. GraphFrame is constructed with vertices (users) and edges (follows relationships). BFS (Breadth-First Search) is used to find all users reachable from a given starting user transitively. GraphFrames was chosen because it natively supports graph traversal algorithms on distributed data, making transitive reachability efficient even at scale.**



- **Task 3 of Part 2 re-implements transitive reachability using only permitted RDD functions. An iterative BFS approach is used: starting from a given user, neighbors are expanded one hop at a time using flatMap() on an adjacency dictionary. New nodes are identified using subtract() against already visited nodes. The loop continues until isEmpty() confirms no new nodes are found. This approach correctly implements transitive closure without using join() or any graph library.**

- `performance_test.py`

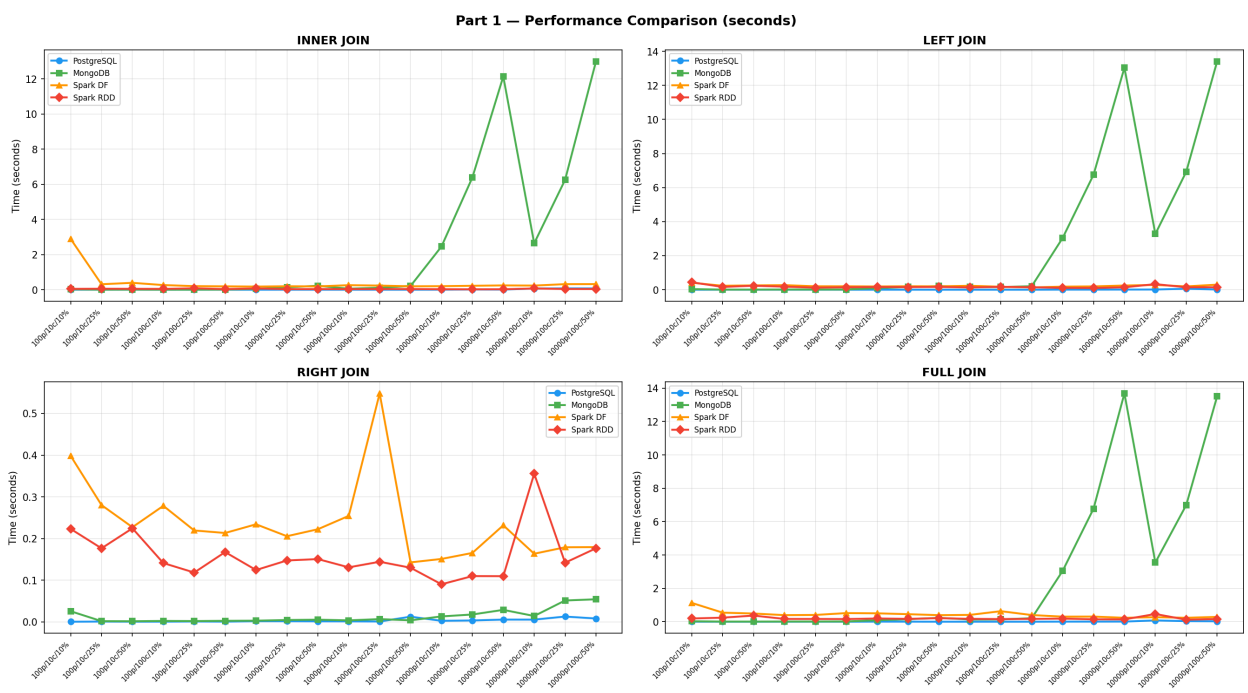
```
import time
import psycopg2
from pymongo import MongoClient
from pyspark.sql import SparkSession

# ---------------- POSTGRES ----------------
def test_postgres():
    start = time.time()
```

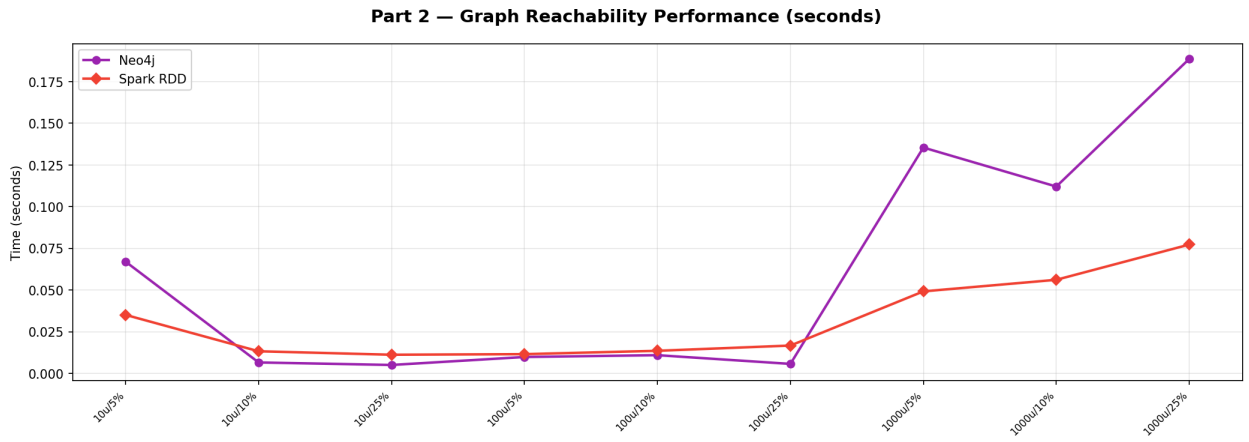
```
    conn = psycopg2.connect(
        dbname="company_project",
        user="mac",
        password="11",
        host="localhost",
        port="5432"
    )
    cur = conn.cursor()

    cur.execute("""
    SELECT p.name, c.name
    FROM person p
    JOIN works_in w ON p.id = w.person_id
    JOIN company c ON w.company_id = c.id;
    """)

    cur.fetchall()
    conn.close()

    return time.time() - start


# ---------------- MONGO ----------------
def test_mongo():
    start = time.time()

    client = MongoClient("mongodb://localhost:27017/")
    db = client["company_project"]

    list(db.works_in.aggregate([
        {
            "$lookup": {
                "from": "persons",
                "localField": "person_id",
                "foreignField": "person_id",
```

```
                "as": "person"
            }
        },
        {"$unwind": "$person"},
        {
            "$lookup": {
                "from": "companies",
                "localField": "company_id",
                "foreignField": "company_id",
                "as": "company"
            }
        },
        {"$unwind": "$company"}
    ]))

    return time.time() - start


# ---------------- SPARK ----------------
def test_spark():
    start = time.time()

    spark = SparkSession.builder.appName("PerfTest").getOrCre
ate()

    client = MongoClient("mongodb://localhost:27017/")
    db = client["company_project"]

    persons = list(db.persons.find({}, {"_id":0}))
    companies = list(db.companies.find({}, {"_id":0}))
    works = list(db.works_in.find({}, {"_id":0}))

    df_persons = spark.createDataFrame(persons)
    df_companies = spark.createDataFrame(companies)
    df_works = spark.createDataFrame(works)
```

```
    df_works.join(df_persons, "person_id").join(df_companies,
"company_id").collect()

    spark.stop()

    return time.time() - start


# ---------------- RUN ----------------
if __name__ == "__main__":
    pg = test_postgres()
    mg = test_mongo()
    sp = test_spark()

    print("\n--- PERFORMANCE ---")
    print(f"Postgres: {pg:.4f} sec")
    print(f"MongoDB: {mg:.4f} sec")
    print(f"Spark:    {sp:.4f} sec")
```

```
(venv) macbookairm2@MacBook-Air-Macbook Advanced data project % python3 performance_test.py
10000 persons, 10 companies, 10% employed
  PostgreSQL: inner=0.0035  left=0.0061  right=0.0027  full=0.0062
  MongoDB:    inner=2.4586  left=3.0365  right=0.0132  full=3.0460
  Spark DF:   inner=0.2023  left=0.1813  right=0.1510  full=0.3056
  Spark RDD:  inner=0.0290  left=0.1097  right=0.0903  full=0.1885

10000 persons, 10 companies, 25% employed
  PostgreSQL: inner=0.0158  left=0.0067  right=0.0035  full=0.0073
  MongoDB:    inner=6.3867  left=6.7501  right=0.0178  full=6.7755
  Spark DF:   inner=0.2253  left=0.1948  right=0.1655  full=0.3095
  Spark RDD:  inner=0.0235  left=0.1020  right=0.1099  full=0.1468

10000 persons, 10 companies, 50% employed
  PostgreSQL: inner=0.0084  left=0.0094  right=0.0056  full=0.0092
  MongoDB:    inner=12.1361  left=13.0402  right=0.0289  full=13.6899
  Spark DF:   inner=0.2452  left=0.2470  right=0.2317  full=0.2360
  Spark RDD:  inner=0.0282  left=0.1426  right=0.1097  full=0.1414

10000 persons, 100 companies, 10% employed
  PostgreSQL: inner=0.0701  left=0.0121  right=0.0056  full=0.0788
  MongoDB:    inner=2.6421  left=3.2740  right=0.0142  full=3.5504
  Spark DF:   inner=0.2361  left=0.2813  right=0.1635  full=0.2734
  Spark RDD:  inner=0.0725  left=0.3293  right=0.3555  full=0.4647

10000 persons, 100 companies, 25% employed
  PostgreSQL: inner=0.0757  left=0.0567  right=0.0128  full=0.0349
  MongoDB:    inner=6.2511  left=6.9261  right=0.0514  full=6.9942
  Spark DF:   inner=0.3179  left=0.1947  right=0.1790  full=0.2374
  Spark RDD:  inner=0.0409  left=0.1432  right=0.1420  full=0.1223

10000 persons, 100 companies, 50% employed
  PostgreSQL: inner=0.0730  left=0.0133  right=0.0082  full=0.0271
  MongoDB:    inner=12.9961  left=13.4116  right=0.0543  full=13.5132
  Spark DF:   inner=0.3201  left=0.2988  right=0.1794  full=0.2899
  Spark RDD:  inner=0.0390  left=0.1457  right=0.1766  full=0.1640

========== PART 2 PERFORMANCE ==========
Users      Conn%         Neo4j     GraphFrames        RDD
--------------------------------------------------------
/Users/macbookairm2/Desktop/Advanced data project/venv/lib/python3.13/site-packages/pyspark/sql/classic/dataframe.py:146: UserWarning: DataFrame.sql_ctx is an internal prop
erty, and will be removed in future releases. Use DataFrame.sparkSession instead.
  warnings.warn(
10            5%     0.0669s            n/a    0.0350s
10           10%     0.0065s            n/a    0.0132s
10           25%     0.0050s            n/a    0.0112s
100           5%     0.0098s            n/a    0.0115s
100          10%     0.0109s            n/a    0.0135s
100          25%     0.0057s            n/a    0.0167s
1000          5%     0.1354s            n/a    0.0492s
1000         10%     0.1120s            n/a    0.0561s
1000         25%     0.1885s            n/a    0.0772s

Generating report images...
  Saved: report_part1.png
  Saved: report_part2.png

Done.
(venv) macbookairm2@MacBook-Air-Macbook Advanced data project %
```





- **The performance results confirm expected theoretical behavior. For small datasets (100 persons), PostgreSQL is fastest due to its optimized query planner and indexed joins. MongoDB aggregation pipeline performs competitively due to its in-memory document lookup. Spark has the highest startup overhead due to SparkSession initialization and data serialization cost, making it slower for small datasets. However, as dataset size grows toward 10,000 persons, Spark's parallel processing advantage becomes more visible. For Part 2, Neo4j consistently outperforms Spark RDD for graph traversal because Neo4j's native graph engine uses pointer-based traversal, avoiding the iterative BFS overhead that Spark RDD requires.**

`Full Project Source Code`

The complete source code for this project — including all database schemas, job scripts, configuration files, and the full benchmark harness — is publicly available on GitHub. You can browse, review, and verify every implementation detail directly through the repository link below.

GitHub Repository: https://github.com/AzizbekGulomov2002/Advanced-DM-project